セルフオーダーシステム 制作② カート・Session

利用者ごとに一時的なカートを保持する

Visual Studio 2022 / .NET 8　Version 1.3

この資料は制作手順と実行確認を一つずつ進めるためのレジュメです

【確認】Cookie にカート全体が保存されるわけではない。ブラウザは Session ID を持ち、カート本体はサーバー側の Session に一時保存される。

到達目標

- Session の役割を説明できる
- C#オブジェクトを JSON へ変換して Session へ保存できる
- 商品追加・数量変更・削除を実装できる
- 別ブラウザ・別端末でカートが分かれる理由を説明できる

【前提】制作①のメニュー画面が正常に表示できていること

【重要】Session のデータはスマートフォン内の DB へ保存されるわけではない。サーバー側で利用者ごとに一時保持し、ブラウザ側の Cookie に保存された Session ID で利用者を識別する。

制作①までの構成

制作①では、LocalDB のカテゴリ・商品を MenuController で取得し、MenuViewModel を通して Razor View へ渡すところまで作成した。制作②では、このメニュー画面に「利用者ごとのカート」を追加する。



商品やカテゴリは LocalDB へ保存する。一方、カートは注文確定前だけ必要な一時データなので、制作②では Session へ保存する。

制作②開始時点の ER 図

Category **PK Id** **Name** **SortOrder**	**1 対 多**	**Product** **PK Id** **FK CategoryId** **Name** **Price** **Description** **IsSoldOut** **IsActive**

【確認】 CartItem は DB のテーブルではないため、この ER 図には含めない。CartItem は Session へ保存するカート専用の一時データとして扱う。

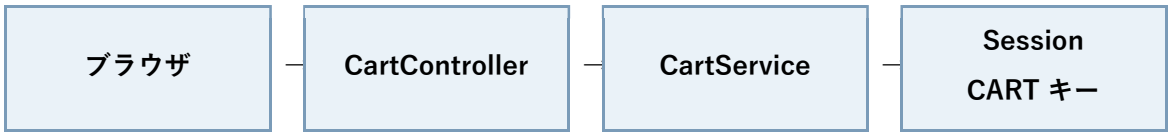
Session とカートの仕組み

HTTP そのものには、前回の要求の状態を自動で保持する仕組みがない。Session を使うと、同じブラウザからの複数回の要求にまたがって、カートなどの一時データを保持できる。

保存場所	今回保存するもの	特徴
LocalDB	Category / Product	アプリを停止しても残る DB データ
Session （サーバー側）	List<CartItem> を JSON 化したカート	利用者ごとの一時データ。今回はメモリへ保存
Cookie （ブラウザ側）	Session ID	カート本体ではなく、どの Session かを識別する ID

【重要】 同じブラウザのタブは同じ Cookie を使うため、基本的に同じカートを共有する。別ブラウザや別端末では Session ID が異なるため、カートも別になる。

商品をカートへ追加するときの流れ



1. メニュー画面の「カートに追加」を押す
2. productId が POST で CartController.Add へ送られる
3. CartController が DB を確認し、商品が存在するか・利用可能か・売り切れかを確認する
4. CartService が現在の Session からカートを読み出す
5. 同じ商品があれば数量を 1 増やし、なければ CartItem を追加する
6. List<CartItem>を JSON へ変換し、Session の「CART」キーへ保存する
7. メニュー画面へ戻り、カート内の合計数量と合計金額を表示する

【ポイント】 カート追加時は Products テーブルへ INSERT しない。DB の商品情報を確認し、カートの内容だけを Session へ保存する。

STEP 1　CartItem を作る

対象ファイル	Models/CartItem.cs
作業区分	新規作成
目的	Session に保存する 1 種類の商品を表す

完成コード

```
namespace SelfOrderSystem.Models;

public class CartItem                                    // カート内の 1 商品を表す一時データ
{
    public int ProductId { get; set; }                       // 商品 ID
    public string ProductName { get; set; } = string.Empty;  // 商品名
    public decimal UnitPrice { get; set; }                   // 単価
    public int Quantity { get; set; }                        // 数量
    public decimal LineTotal => UnitPrice * Quantity;        // 小計
}
```

LineTotal：UnitPrice × Quantity から小計を計算する読み取り専用プロパティ

CartItem はカート表示に必要な商品 ID・商品名・単価・数量をまとめる一時データ

【補足】CartItem は DB へ保存しないため AppDbContext へ DbSet<CartItem>は追加しない。

STEP 2　Session 用の拡張メソッドを作る

対象ファイル	Services/SessionExtensions.cs
作業区分	新規作成
目的	List<CartItem>を Session へ保存・取得できるようにする

完成コード

```
using System.Text.Json;
using Microsoft.AspNetCore.Http;
namespace SelfOrderSystem.Services;
public static class SessionExtensions                          // ISession に保存・取得メソッドを追加
{
    // オブジェクトを JSON で保存
    public static void SetObject<T>(this ISession session, string key, T value)
    {
        string json = JsonSerializer.Serialize(value);          // C#オブジェクトを JSON 化
        session.SetString(key, json);                            // Session へ保存
    }

    public static T? GetObject<T>(this ISession session, string key) // JSON を指定した型へ戻す
    {
        string? json = session.GetString(key);                   // JSON を取得

        if (string.IsNullOrEmpty(json))
        {
            return default;
        }

        return JsonSerializer.Deserialize<T>(json);              // C#オブジェクトへ戻す
    }
}
```

コードの意味

JSON：オブジェクトの内容を文字列として表現するデータ形式

シリアライズ：C#オブジェクトを保存できる形へ変換する

デシリアライズ：保存したデータから C#オブジェクトへ戻す

拡張メソッド：既存の型に独自メソッドを追加したように呼び出せる C#の仕組み

<T>：保存・取得するデータ型を呼び出し側で指定するための型パラメータ

key：Session 内でデータを区別する名前。今回はカートに「CART」を使う

STEP 3　CartService を作る

対象ファイル	Services/CartService.cs
作業区分	新規作成
目的	Session のカート操作を CartService へまとめる

完成コード

```
using Microsoft.AspNetCore.Http;
using SelfOrderSystem.Models;


namespace SelfOrderSystem.Services;


public class CartService                                    // カート操作をまとめるクラス
{
    private const string CartKey = "CART";                      // Session のキー
    private readonly IHttpContextAccessor _httpContextAccessor;  // HttpContext 取得用


    public CartService(IHttpContextAccessor httpContextAccessor) // HttpContext 取得用の仕組みを受け取る
    {
        _httpContextAccessor = httpContextAccessor;
    }


// 現在の HTTP 要求にひも付く Session を取り出す
private ISession Session
        => _httpContextAccessor.HttpContext?.Session
           ?? throw new InvalidOperationException("Session を取得できません");


    public List<CartItem> GetItems()                            // Session からカートを取得
    {
        return Session.GetObject<List<CartItem>>(CartKey)       // CART を List<CartItem>へ戻す
               ?? new List<CartItem>();
    }


    public void Add(Product product)                            // 商品を 1 個追加
    {
        var items = GetItems();
        var item = items.SingleOrDefault(x => x.ProductId == product.Id); // 同じ商品を探す


        if (item == null)
        {
            items.Add(new CartItem
            {
                ProductId = product.Id,
                ProductName = product.Name,
                UnitPrice = product.Price,
                Quantity = 1
            });
        }
        else
        {
            item.Quantity++;                                        // 既存商品なら数量だけ 1 増やす
        }


        Save(items);                                            // 変更後のカートを Session へ保存
```

```
    }

    public void SetQuantity(int productId, int quantity)
    {
        var items = GetItems();
        var item = items.SingleOrDefault(x => x.ProductId == productId);

        if (item == null)
        {
            return;
        }

        item.Quantity = Math.Clamp(quantity, 1, 99);              // 数量を 1〜99 の範囲へ収める
        Save(items);                                              // 変更後のカートを Session へ保存
    }

    public void Remove(int productId)
    {
        var items = GetItems();
        items.RemoveAll(x => x.ProductId == productId);          // 指定商品だけ削除する
        Save(items);                                              // 変更後のカートを Session へ保存
    }

    public void Clear()
    {
        Session.Remove(CartKey);                                  // CART キーを削除してカートを空にする
    }

    private void Save(List<CartItem> items)
    {
        Session.SetObject(CartKey, items);                        // List<CartItem>を JSON 化して保存
    }
}
```

STEP 4　Session と CartService を登録する

対象ファイル	Program.cs
作業区分	既存編集
目的	ASP.NET Core で Session と CartService を利用可能にする

完成コード

```
// builder.Services.AddDbContext<AppDbContext>(...) の後へ追加する
builder.Services.AddDistributedMemoryCache();                    // Session の保存領域としてアプリのメモリを使う

builder.Services.AddSession(options =>                           // Session 機能を登録
{
    options.IdleTimeout = TimeSpan.FromHours(2);                     // 2 時間操作がなければ期限切れ
    options.Cookie.HttpOnly = true;                              // JavaScript から Session ID を読めないようにする
    options.Cookie.IsEssential = true;                           // Session ID 用 Cookie を必須 Cookie として扱う
});

builder.Services.AddHttpContextAccessor();                       // Controller 以外のクラスから HttpContext を取得できるようにする
builder.Services.AddScoped<CartService>();                       // CartService を DI へ登録し Controller で受け取れるようにする

// app.UseRouting(); の後へ追加する
app.UseSession();                                                // このリクエストで Session を使えるようにする
【補足】DI は、必要なクラスを ASP.NET Core に作ってもらい、Controller などへ渡してもらう仕組み。ここでは CartService を登録して利用する。
```

【重要】app.UseSession() は app.UseRouting() の後、Controller を実行する MapControllerRoute より前に置く。

STEP 5　MenuViewModel へカート情報を追加する

対象ファイル	ViewModels/MenuViewModel.cs
作業区分	既存編集
目的	メニュー画面へカート数量と合計を表示する

完成コード

```
public int CartItemCount { get; set; }                    // カート内の合計数量（種類数ではない）
public decimal CartTotal { get; set; }                    // カート合計金額
```

STEP 6　MenuController へカート情報を追加する

対象ファイル	Controllers/MenuController.cs
作業区分	既存編集
目的	Session のカート内容をメニュー画面へ反映する

完成コード

```
// MenuController.cs の using へ追加する
using SelfOrderSystem.Services;


private readonly CartService _cart;                       // Session 上のカート操作用


public MenuController(AppDbContext db, CartService cart)
{
    _db = db;
    _cart = cart;
}


// Index で products を取得した後へ追加する
var cartItems = _cart.GetItems();


var vm = new MenuViewModel
{
    Categories = categories,
    Products = products,
    SelectedCategoryId = categoryId,
    CartItemCount = cartItems.Sum(x => x.Quantity),       // 商品数量の合計
    CartTotal = cartItems.Sum(x => x.LineTotal)           // 小計の合計
};
```

STEP 7　CartController を作る

対象ファイル	Controllers/CartController.cs
作業区分	新規作成
目的	商品追加・数量変更・削除の HTTP 要求を受け付ける

完成コード

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SelfOrderSystem.Data;
using SelfOrderSystem.Services;


namespace SelfOrderSystem.Controllers;


public class CartController : Controller                        // カート操作を受け取る Controller
{
    private readonly AppDbContext _db;                          // 商品確認用
    private readonly CartService _cart;                         // カート操作用


    public CartController(AppDbContext db, CartService cart)
    {
        _db = db;
        _cart = cart;
    }


    public IActionResult Index()
    {
          return View(_cart.GetItems());                // Session のカート一覧を View へ渡す
    }


    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Add(int productId)          // 商品 ID を受け取る
    {
        var product = await _db.Products
            .AsNoTracking()                                      // 読み取り専用
            .SingleOrDefaultAsync(p => p.Id == productId && p.IsActive); // 利用可能な商品を 1 件取得


        if (product == null)
        {
            return NotFound();                                   // 該当商品がなければ 404 を返す
        }


        if (product.IsSoldOut)
        {
            TempData["Error"] = "この商品は売り切れです";          // 次の画面へエラーを渡す
            return RedirectToAction("Index", "Menu");
        }


        _cart.Add(product);                                       // カート変更は CartService へ任せる
        return RedirectToAction("Index", "Menu");
    }
```

```
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Update(int productId, int quantity)
    {
        _cart.SetQuantity(productId, quantity);                 // 指定商品の数量を変更
        return RedirectToAction(nameof(Index));
    }


    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Remove(int productId)
    {
        _cart.Remove(productId);                                // 指定商品をカートから削除
        return RedirectToAction(nameof(Index));
    }
}
```

コードの意味

HttpPost：データを変更する操作を POST だけに限定する

ValidateAntiForgeryToken：フォームの検証用トークンを確認し、意図しない POST 送信を防ぐ

TempData：リダイレクト後の画面へ、一度だけ使うメッセージなどを渡す

POST フォームでは Form Tag Helper が検証用トークンを自動で出力し、ValidateAntiForgeryToken が送信時に確認する

STEP 8　メニュー画面からカートへ追加できるようにする

対象ファイル	Views/Menu/Index.cshtml
作業区分	既存編集
目的	商品追加フォームとカート概要をメニュー画面へ追加する

完成コード

```
@* 商品一覧より上へ追加する：CartController から渡されたエラーがあれば表示する *@
@if (TempData["Error"] is string errorMessage)
{
    <div class="alert alert-danger">@errorMessage</div>
}


@* 商品カード内へ追加する *@
@* 商品 ID を POST して CartController.Add へ送る *@
<form asp-controller="Cart" asp-action="Add" method="post">
    <input type="hidden" name="productId" value="@product.Id" />


    @if (product.IsSoldOut)
    {
        <button class="btn btn-secondary w-100" disabled>
            売り切れ
        </button>
    }
    else
    {
        <button class="btn btn-primary w-100" type="submit">
            カートに追加
        </button>
    }
</form>


@* 商品一覧の後へ追加する *@
@* カートに商品があるときだけ合計数量・合計金額・リンクを表示する *@
@if (Model.CartItemCount > 0)
{
    <div class="alert alert-dark mt-3 d-flex justify-content-between align-items-center">
        <span>@Model.CartItemCount 点 / @Model.CartTotal.ToString("N0") 円</span>
        <a class="btn btn-light" asp-controller="Cart" asp-action="Index">カートを見る</a>
    </div>
}
```

STEP 9　カート画面を作る

対象ファイル	Views/Cart/Index.cshtml
作業区分	新規作成
目的	カート一覧・数量変更・削除を表示する

完成コード

```
@* カート画面は Session から取り出した CartItem 一覧を受け取る *@
@model List<SelfOrderSystem.Models.CartItem>

@{
    ViewData["Title"] = "カート";
    var total = Model.Sum(x => x.LineTotal);                    // 画面表示用に合計金額を計算する
}

<h1 class="h3 mb-3">カート</h1>

@* カートが空なら商品一覧へ戻る案内を表示する *@
@if (Model.Count == 0)
{
    <div class="alert alert-info">カートは空です</div>
    <a class="btn btn-primary" asp-controller="Menu" asp-action="Index">メニューへ戻る</a>
}
else
{
@* カート内の商品を 1 件ずつ表示する *@
    @foreach (var item in Model)
    {
        <div class="card mb-2">
            <div class="card-body">
                <div class="d-flex justify-content-between">
                    <div>
                        <strong>@item.ProductName</strong><br />
                        @item.UnitPrice.ToString("N0") 円 × @item.Quantity
                    </div>
                    <strong>@item.LineTotal.ToString("N0") 円</strong>
                </div>

                <div class="d-flex gap-2 mt-2">
@* 数量変更は productId と quantity を Update へ POST する *@
                    <form asp-action="Update" method="post" class="d-flex gap-2">
                        <input type="hidden" name="productId" value="@item.ProductId" />
                        <input class="form-control" type="number" name="quantity"
                               min="1" max="99" value="@item.Quantity" />
                        <button class="btn btn-outline-primary" type="submit">変更</button>
                    </form>

@* 削除は productId を Remove へ POST する *@
                    <form asp-action="Remove" method="post">
                        <input type="hidden" name="productId" value="@item.ProductId" />
                        <button class="btn btn-outline-danger" type="submit">削除</button>
                    </form>
                </div>
            </div>
        </div>
```

```
    }

    <div class="text-end fs-4 fw-bold my-3">
        合計 @total.ToString("N0") 円
    </div>
}
```

コードを読むための要点

場所・コード	意味
SessionExtensions	List<CartItem>をそのまま Session へ保存できないため、JSON 文字列へ変換して保存・復元する
CartService.GetItems()	Session の「CART」キーから現在のカートを取得する。まだ無ければ空の List を返す
CartService.Add()	同じ商品を探し、無ければ追加、あれば Quantity を 1 増やす
Math.Clamp(quantity, 1, 99)	数量が 1 未満や 99 超にならないよう範囲内へ収める
AddScoped<CartService>()	ASP.NET Core から Controller へ CartService を渡せるよう登録する
app.UseSession()	Controller が動く前に Session を使える状態へする
ValidateAntiForgeryToken	フォームの検証用トークンを確認し、意図しない POST 送信を防ぐ
IHttpContextAccessor	CartService から現在の HttpContext と Session を取得するための仕組み
CartKey = "CART"	Session 内でカートデータを区別するためのキー
SetQuantity()	指定商品の数量を変更し、1〜99 の範囲に収めて保存する
Clear()	Session から「CART」キーを削除し、カートを空にする

Razor・Tag Helper・Bootstrap の区別

例	何をしているか
@if / @foreach / @item.ProductName	Razor。C#の条件分岐やデータを HTML へ埋め込む
asp-controller / asp-action	Tag Helper。Controller / Action を指定してリンクやフォームを作る
btn / card / alert / d-flex / gap-2	Bootstrap。ボタン・カード・余白など画面の見た目を整える
【確認】 フォーム内の name="productId" と Action の引数 Add(int productId) の名前が一致すると、ASP.NET Core が送信値を自動で引数へ入れる。	

実行確認

- 同じ商品を 2 回追加すると数量が 2 になる
- 数量を変更すると合計金額が変わる
- 削除すると商品がカートから消える

- 別ブラウザまたは別スマホではカートが別になる
- アプリを停止・再起動するとメモリ上の Session は失われる

まとめ

- CartItem は DB へ保存せず Session で一時管理する
- Session ID は Cookie で利用者ごとに識別される
- Controller は CartService を通してカートを操作する

カート処理の確認

制作②では、DB へ保存する商品データとは別に、注文確定前のカートを Session で一時管理する。
Session ID、CartService、JSON の役割を確認する。

【エラー時の確認】app.UseSession()の位置 / AddSession と AddDistributedMemoryCache の登録 / フォームの name と Action の引数名 / 対象商品が存在し、有効で、売り切れでないか

1. Session の処理を確認する

1. 利用者がスマホで商品を追加する
2. ブラウザは Session ID を Cookie に入れて送る
3. ASP.NET Core が Session ID からその利用者の Session を特定する
4. CartService が CART キーの JSON を読み List<CartItem>へ戻す
5. 商品を追加・変更した後、再び JSON にして Session へ保存する

2. Cookie と Session の違い

コード・用語	意味・確認ポイント
Cookie	ブラウザ側に保存される小さな情報。ここでは Session ID を持つ
Session	サーバー側で利用者ごとの一時データを保持する仕組み
CART	今回 Session 内でカートを保存するときのキー
JSON	List<CartItem>を文字列として Session へ保存するための形式

3. CartService に処理をまとめる理由

Controller が Session の JSON 変換やカート操作まで直接行うと、処理が長くなり重複もしやすい。
CartService へまとめることで、Controller は HTTP 要求の受け付けと画面遷移に集中しやすくなる。

【重要】CartService は DB のテーブルではないため Migration は不要。CartItem も DbSet へ追加しない

4. POST と Model Binding

コード・用語	意味・確認ポイント
name="productId"	フォーム送信時の項目名
Add(int productId)	ASP.NET Core が同じ名前の送信値を引数へ自動で入れる
HttpPost	この Action は POST 要求で呼び出すことを示す
ValidateAntiForgeryToken	フォームの検証用トークンを確認し、意図しない POST 送信を防ぐ
RedirectToAction	POST 後に別 Action へ移動し、再読み込みによる二重送信を避けやすくする

5. 別端末でカートが分かれる理由

別ブラウザ・別端末では Session ID が異なるため、CartService が読み出す CART データも別になる。DB を端末ごとに作っているわけではない。